

PART A: Theoretical Questions

1. What are the four main factors affecting the performance of a program?

1. **Algorithm** – How the task is solved. A better algorithm runs faster.
2. **Programming Language, Compiler, and Architecture** – These affect how many instructions are needed.
3. **Processor and Memory System** – They affect how fast instructions are executed.
4. **I/O System** – Handles data input/output speed, affecting total performance.

2. Which factors are software-based and which are hardware-based?

- **Software-based:** Algorithm, Programming Language, Compiler
- **Hardware-based:** Processor, Memory System, I/O System

3. Explain with an example how the algorithm affects performance.

- Sorting with **Bubble Sort** (slow) vs **Quick Sort** (fast). Both give correct results, but Quick Sort is much faster for large data.

4. Explain “Use abstraction to simplify design” with a real-life coding example.

- In C programming: `int x = 5;` stores 5 in memory. You don't need to know how memory stores 5 as binary or voltage; abstraction hides this complexity.

5. Why is "Make the common case fast" a good design principle?

- Most time is spent on common tasks. If you speed up common cases, overall performance improves.
- Example: Optimizing webpage image loading is better than optimizing rare print tasks.

6. What is “Performance via parallelism”? Give an example.

- Doing many tasks at once.
- Example: Multi-core processors running different apps or threads at the same time.

7. What is “Performance via pipelining”?

- A technique to execute multiple instructions in overlapping stages. Like an assembly line in a factory.

8. What is “Performance via prediction”? Explain with a branch prediction example.

- CPU guesses next steps to avoid waiting.
- Example: In `if(x > 10)`, CPU guesses which block will run and executes early. If wrong, it undoes and corrects.

9. What is the memory hierarchy? Why is it needed?

- Uses fast & expensive memory (like cache) and slow & cheap memory (like HDD).
- Balances cost, size, and speed.

10. What is redundancy in system design? How does it help with dependability?

- Adding backup parts to avoid failure.
 - Example: RAID 1 uses 2 hard drives with same data. If one fails, the other continues.
-

PART B: Numerical and Formula-Based Questions

11. Cost and Yield Formulas:

1. Cost per die = Cost per wafer / (Dies per wafer × Yield)
2. Dies per wafer $\approx$ Wafer area / Die area
3. Wafer area = $\pi \times r^2$
4. Die area = height × width
5. Yield = $1 / (1 + (\text{defects per area} \times \text{die area}))^N$

12. Define CPU Time, User CPU Time, and System CPU Time.

- **CPU Time:** Time CPU spends on a task (excluding I/O).
- **User CPU Time:** Time CPU spends running user program code.
- **System CPU Time:** Time CPU spends on OS code related to that task.

13. Define Response Time and Throughput. Give examples.

- **Response Time:** Time to complete one task (e.g., opening a file).
- **Throughput:** Number of tasks completed per time (e.g., 100 files printed/minute).

14. When does replacing a processor improve both response time and throughput?

- When a faster processor is used. Both time per task and total completed tasks increase.

15. When does it only improve throughput?

- Adding more processors to work in parallel. More tasks complete, but individual task time might not reduce.

16. Clock & Performance Formulas

1. Performance = $1 / \text{Execution Time}$
 2. CPU Time = Clock Cycles × Clock Cycle Time
 3. Clock Cycle Time = $1 / \text{Clock Rate}$
 4. CPU Time = (Instruction Count × CPI) / Clock Rate
-

PART C: Definitions & Short Answers

17. What is the SPEC benchmark?

- A set of standard programs used to measure CPU performance.

18. How is SPEC Ratio calculated?

- SPEC Ratio = Reference Time / CPU Time

19. Why do we use geometric mean instead of arithmetic mean?

- Geometric mean handles ratios fairly, avoiding skew from very high/low values.

20. State Amdahl's Law and explain it.

- Speedup is limited by the part that can't be improved.
- Formula: $\text{Speedup} = 1 / ((1 - P) + (P / S))$
- Example: If 80% of code is made 2x faster, total speedup < 2x.

21. Connect Amdahl's Law to a design principle.

- Principle: "Make the common case fast"
- If the common case is optimized, it helps more due to Amdahl's Law.

22. What is ISA?

- ISA (Instruction Set Architecture) is the interface between software and hardware – defines what instructions the CPU can run.

23. What is MIPS?

- MIPS = Millions of Instructions Per Second
- 1 MIPS = 1 million instructions executed per second.

24. When do we need average CPI?

- When a program uses multiple instruction types with different CPI.

25. Define:

- **Clock Period:** Time for one cycle.
 - **Clock Rate:** Number of cycles per second.
 - **CPI:** Clock cycles per instruction.
-

PART D: Problem Solving

26. Compare Computer A and B:

If A = 250 ps & 2.0 CPI, B = 500 ps & 1.2 CPI: - A Time = $I \times 2.0 \times 250 = I \times 500$ ps - B Time = $I \times 1.2 \times 500 = I \times 600$ ps - A is faster by $600/500 = \mathbf{1.2x}$

27. Code Sequence Comparison:

Instr. Type	CPI	Seq1	Seq2
Add	1	2	4
Sub	2	1	1
Mul	3	2	1

- Seq1 Instructions: 5; Seq2: 6
- Seq1 Cycles = $2 \times 1 + 1 \times 2 + 2 \times 3 = 10$
- Seq2 Cycles = $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$
- CPI1 = $10/5 = 2.0$; CPI2 = $9/6 = 1.5$
- **Seq2 is faster despite more instructions**